TAC-UFLD: the mathematics behind every part of the pipeline

A reference for the article and its defence, mapped to the code

Tiago Rodrigues — UTFPR / Residência em IA

companion of the repository `tac-ufld-lane-detection`, 2026-10-19

Abstract

Each section states the model or procedure as equations, names the file and function that implement it, and records how the implementation was checked (unit test, numerical measurement, or derivation). Symbols are defined once in Section 1. Results quoted as *pilot* come from 1 seed and 4 epochs and are indicative only. **Resumo:** cada seção apresenta as equações, o arquivo/função que as implementa e como a implementação foi verificada.

Contents

1 Notation and data **1**

2 Target encoding and decoding **2**

3 Geometric augmentation **2**

4 Networks **2**
4.1 UFLD v1 (baseline) 2
4.2 Light model 3

5 Temporal fusion modules **3**

6 Losses **4**

7 Evaluation metrics **4**

8 Output-level Bayesian filtering **5**

9 Ego-lane geometry, lane change, LDW and LKA **5**

10 Robustness protocols **6**

11 Statistics **6**

12 Cost **6**

13 Planned: distance and time to collision **6**

1 Notation and data

An input frame $I_t \in [0,1]^{3\times H\times W}$ (model size $H\times W = 384\times 512$ for ELAS) at time index t; a clip $\mathcal{C}_t = (I_{t-(T-1)s}, \ldots, I_{t-s}, I_t)$ with T frames spaced s apart (ELAS: $T=3$, $s=2$ at 29.97 Hz, i.e. 67 and 133 ms of history). L lane slots (ELAS: $L=2$, ego-left and ego-right), A row anchors, G horizontal grid cells.

Row anchors. $y_a = \ell(H-1) + \frac{a}{A-1}(u-\ell)(H-1)$, $a = 0, \ldots, A-1$, with the band $[\ell, u] = [0.59, 0.995]$ covering every ELAS region of interest (ROI).
Code: `data/targets.py: make_row_anchors`

ELAS geometry. Each annotated boundary has four x values at the fixed rows $y_k = y_{\text{roi}} + f_k h_{\text{roi}}$, $f_k \in \{0, \frac{1}{4}, \frac{1}{2}, 1\}$; the lane is the polyline through them, linearly interpolated between rows.
Verified: a straight-line fit through the four points has a median maximum residual of 1.0–4.6 px with these fractions versus 15.8–22.5 px with thirds, on all 22 scenes (`tests/test_elas.py`); the lane-centre label `laneCenter` equals the mean of the two bottom points to 0.00 px median over 15,424 frames.

2 Target encoding and decoding

Encoding. For slot l and anchor a with the lane at model column $x_{a,l} \in [0, W-1]$,

$$c_{a,l} = \text{clip}\left(\text{round}\left(\tfrac{x_{a,l}}{W-1}(G-1)\right), 0, G-1\right), \tag{1}$$

$c_{a,l} = G$ ("no lane") where the slot is known but the lane does not cross the anchor, and $c_{a,l} = \varnothing$ (ignored) outside the annotated band or for an unknown slot.
Code: `data/targets.py: encode_targets, x_to_bin`

Decoding (official UFLD). With logits $\boldsymbol{z}_{a,l} \in \mathbb{R}^{G+1}$,

$$p_{a,l}(k) = \text{softmax}(\boldsymbol{z}_{a,l})_k, \qquad \hat{e}_{a,l} = 1 - p_{a,l}(G) \quad \text{(existence)}, \tag{2}$$

$$q_{a,l}(i) = \text{softmax}(\boldsymbol{z}_{a,l,0:G-1})_i, \qquad \hat{\imath}_{a,l} = \mathbb{E}_q[i] = \textstyle\sum_{i=0}^{G-1} i\, q_{a,l}(i), \qquad \hat{x}_{a,l} = \hat{\imath}_{a,l}\tfrac{W-1}{G-1}. \tag{3}$$

The expectation (soft-argmax) gives sub-cell resolution and is differentiable.
Code: `decoding.py: decode_logits, expected_x`

Predictive variance (added 2026-10-19). The same distribution gives its spread,

$$\sigma^2_{a,l} = \text{Var}_q[i]\left(\tfrac{W-1}{G-1}\right)^2 = \left(\textstyle\sum_i i^2 q(i) - \hat{\imath}^2\right)\left(\tfrac{W-1}{G-1}\right)^2, \tag{4}$$

in model px^2: small for a confident frame, large when the frame is occluded or blurred.
Code: `decoding.py: position_variance; evaluation/predictor.py (Predictions.x_var)`
Verified: equals $\sum_i q(i)(i - \hat{\imath})^2$ to 10^{-5} and is 0 for a one-hot distribution (`test_position_variance_matches_the_categorical_formula`).

3 Geometric augmentation

One homography $\boldsymbol{H} \in \mathbb{R}^{3\times3}$ per clip (rotation, scale, translation, crop-zoom, perspective jitter of the corners): every frame is warped by $\boldsymbol{H}$, so the motion between frames is kept, and the targets are re-encoded from the transformed polyline vertices $\tilde{\boldsymbol{p}} = \pi(\boldsymbol{H}[\boldsymbol{p}; 1])$ (π = dehomogenisation; a homography maps segments to segments, so transforming the vertices is exact). A transformed row may carry "no lane" only if its visible content lies inside the source ROI, tested by mapping back with $\boldsymbol{H}^{-1}$.
Code: `data/geometric.py: GeometricAugmenter.sample, encode_targets_warped`

4 Networks

4.1 UFLD v1 (baseline)

ResNet-18 features $\boldsymbol{F} \in \mathbb{R}^{512\times h\times w}$, $h = \lceil H/32 \rceil$, $w = \lceil W/32 \rceil$; a 1×1 convolution to 8 channels; flattening $\boldsymbol{f} \in \mathbb{R}^{8hw}$; a two-layer head

$$\boldsymbol{z} = \boldsymbol{W}_2\,\text{ReLU}(\boldsymbol{W}_1\boldsymbol{f} + \boldsymbol{b}_1) + \boldsymbol{b}_2 \in \mathbb{R}^{(G+1)AL}, \quad \boldsymbol{W}_1 \in \mathbb{R}^{2048\times8hw}. \tag{5}$$

The head sees the whole image (global context). Parameters at 384×512, $A = 18$, $G = 100$, $L = 2$: head $8 \cdot 12 \cdot 16 \cdot 2048 + 2048 \cdot 101 \cdot 36 \approx 10.6$ M, total 21.8 M; 7.12 GMACs.
Code: `models/ufld.py: UFLDNet; models/resnet.py (torchvision layer-for-layer, strict ImageNet load)`

4.2 Light model

Four stride-2 blocks of two 3×3 convolutions (32, 64, 96, 128 channels), output stride 16. Receptive field by the recursion $r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}$, $j_\ell = j_{\ell-1} s_\ell$, $r_0 = j_0 = 1$: $r = 91$ px after the backbone and 123 px after the 3×3 head convolution. Each output cell therefore sees only a local neighbourhood, so the head must read the feature row at the anchor's own height.

Anchor-row pooling (fix of 2026-10-18). Let $\phi_a = (y_a + \frac{1}{2})/H$ be the anchor height as a fraction and $\rho_a = \phi_a h_f - \frac{1}{2}$ its position on the feature grid (h_f rows, centre of row i at $(i + \frac{1}{2})/h_f$). With $i_0 = \lfloor \rho_a \rfloor$ and $\beta = \rho_a - i_0$, the row-interpolation matrix $\boldsymbol{R} \in \mathbb{R}^{A \times h_f}$ has $R_{a,i_0} = 1 - \beta$, $R_{a,i_0+1} = \beta$; the column matrix $\boldsymbol{C} \in \mathbb{R}^{G \times w_f}$ averages the columns of each cell (the bins of adaptive average pooling). The pooled features of channel c are

$$\boldsymbol{P}_c = \boldsymbol{R}\, \boldsymbol{F}_c\, \boldsymbol{C}^\top \in \mathbb{R}^{A \times G}. \tag{6}$$

A per-lane 1×1 projection and one MLP shared by all (a, l) produce $\boldsymbol{z}_{a,l} \in \mathbb{R}^{G+1}$.
Code: `models/lite.py: AnchorRowPool, LanePixelHead`
Verified: input-gradient support of every anchor contains its own image row (anchor 0, $y = 226$: rows 147–285); the former adaptive pooling made anchor 0 read rows 0–77 (`test_aligned_head_reads_each_anchor_row`); pilot held-out lane F1 of the light baseline 0.218 → 0.895.

5 Temporal fusion modules

Per-frame features $\boldsymbol{F}_\tau$, $\tau = 1..T$ (T = current). The fusions added in v0.4 and later (residual gate, ConvGRU, attention, occlusion gate) are initialised so that the warm-started temporal model starts at, or very near, its single-frame baseline; the weighted and gated fusions of the supervisor's notebook start from fixed frame priors instead.

Weighted (UFLD v0.2/v0.4). $\boldsymbol{F}^\star = \sum_\tau w_\tau \boldsymbol{F}_\tau$, $\boldsymbol{w} = \mathrm{softmax}(\boldsymbol{\theta})$, initial $\boldsymbol{w} = (0.1, 0.2, 0.7)$.
Code: `models/ufld.py: TemporalWeightedFusion`

Gated (v0.3). Per-pixel gates $\boldsymbol{g} = \mathrm{softmax}_\tau\big(\mathrm{CNN}([\boldsymbol{F}_1; \ldots; \boldsymbol{F}_T]) + \boldsymbol{b}_0\big)$, $\boldsymbol{F}^\star = \sum_\tau g_\tau \odot \boldsymbol{F}_\tau$.
Code: `models/ufld.py: GatedTemporalFusion`

Warp and residual gate (lite v0.5, UFLD v0.6). A flow $\boldsymbol{u} = \delta \tanh(\mathrm{CNN}([\boldsymbol{F}_\tau; \boldsymbol{F}_T]))$, $|\boldsymbol{u}| \le \delta$ cells; bilinear sampling $\tilde{\boldsymbol{F}}_\tau(\boldsymbol{p}) = \boldsymbol{F}_\tau(\boldsymbol{p} + \boldsymbol{u}(\boldsymbol{p}))$; history $\boldsymbol{F}_h = \mathrm{ReLU}(\mathrm{BN}(\boldsymbol{W}[\tilde{\boldsymbol{F}}_1; \ldots]))$; gate $\gamma = \sigma(\mathrm{CNN}([\boldsymbol{F}_T; \boldsymbol{F}_h]))$ with bias -2 ($\gamma_0 \approx 0.12$); $\boldsymbol{F}^\star = \boldsymbol{F}_T + \gamma \odot (\boldsymbol{F}_h - \boldsymbol{F}_T)$.
Code: `models/lite.py: LearnableFlowWarp, ResidualTemporalFusion`

Caveat. The flow is learned only from the lane loss and may move features vertically by up to δ cells; it partly repaired the head bug before the fix, so the capacity control (history = current frame) is required.

ConvGRU (UFLD v0.7, lite v0.6). With $\boldsymbol{x}_\tau = \mathrm{ReLU}(\mathrm{BN}(\boldsymbol{W}_r \boldsymbol{F}_\tau))$ and $\boldsymbol{h}_0 = \boldsymbol{0}$:

$$\boldsymbol{z}_\tau = \sigma(\boldsymbol{W}_z * [\boldsymbol{x}_\tau; \boldsymbol{h}_{\tau-1}]), \quad \boldsymbol{r}_\tau = \sigma(\boldsymbol{W}_r * [\boldsymbol{x}_\tau; \boldsymbol{h}_{\tau-1}]), \tag{7}$$

$$\tilde{\boldsymbol{h}}_\tau = \tanh(\boldsymbol{W}_h * [\boldsymbol{x}_\tau; \boldsymbol{r}_\tau \odot \boldsymbol{h}_{\tau-1}]), \quad \boldsymbol{h}_\tau = (1 - \boldsymbol{z}_\tau) \odot \boldsymbol{h}_{\tau-1} + \boldsymbol{z}_\tau \odot \tilde{\boldsymbol{h}}_\tau, \tag{8}$$

$$\boldsymbol{F}^\star = \boldsymbol{F}_T + \boldsymbol{W}_o * \boldsymbol{h}_T, \qquad \boldsymbol{W}_o = \boldsymbol{0} \text{ at initialisation.} \tag{9}$$

Code: `models/fusion.py: ConvGRUFusion`

Occlusion-gated write (lite v0.6_occ). A visibility $v_\tau = \sigma(\boldsymbol{W}_v * \boldsymbol{x}_\tau)$ multiplies the update gate: $\boldsymbol{h}_\tau = (1 - \boldsymbol{z}_\tau v_\tau) \odot \boldsymbol{h}_{\tau-1} + \boldsymbol{z}_\tau v_\tau \odot \tilde{\boldsymbol{h}}_\tau$, so a low-visibility frame keeps the memory (OMR, SIGMA-Lane). Initial bias $+3$ ($v_0 \approx 0.95$).
Code: `models/fusion.py: OcclusionGatedConvGRUFusion`

Cross-frame attention (UFLD v0.8, lite v0.7). Queries from the current frame, keys/values from every frame pooled to at most 12×20 cells with a frame embedding: $\mathrm{Attn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top / \sqrt{d_k})\boldsymbol{V}$ (multi-head); read-out added through a zero-initialised projection. Cost $\mathcal{O}(N_q\, T\, N_{kv}\, d)$.
Code: `models/fusion.py: TemporalAttentionFusion`

Temporal shift (lite_tsm). At the input of blocks 2–4 the first $\lfloor C/8 \rfloor$ channels of frame τ are replaced by those of frame $\tau - 1$ (zeros for the first frame): $\mathcal{S}(\boldsymbol{X})_{\tau,c} = \boldsymbol{X}_{\tau-1,c}$ for $c < C/8$, else $\boldsymbol{X}_{\tau,c}$. Causal, zero parameters and MACs; in deployment one slice per shifted block is carried.
Code: `models/lite.py: temporal_shift, LiteTSM`
Verified: carry mode equals the clip forward (`tests/test_lite_tsm_occ.py`); pilot test F1 0.938 (light baseline 0.892, static control 0.931).

6 Losses

For the annotated cells $\Omega = \{(a, l) : c_{a,l} \neq \varnothing\}$, with $p_{a,l} = \mathrm{softmax}(\boldsymbol{z}_{a,l})$:

$$\mathcal{L}_{\text{focal}} = -\tfrac{1}{|\Omega|} \textstyle\sum_{\Omega} (1 - p_{a,l}(c_{a,l}))^{\gamma} \log p_{a,l}(c_{a,l}), \quad \gamma = 2 \quad \text{(official UFLD)}, \tag{10}$$

$$\mathcal{L}_{\text{coord}} = \mathrm{SmoothL1}\big(\hat{x}/(W-1),\, x/(W-1)\big) \text{ on positive cells (v0.4)}, \tag{11}$$

$$\mathcal{L}_{\text{temp}} = \mathrm{SmoothL1}(\boldsymbol{z}^{(t)}_{0:G-1}, \boldsymbol{z}^{(t-1)}_{0:G-1}) + 0.25\,\mathrm{SmoothL1}(e^{(t)}, e^{(t-1)}), \tag{12}$$

$$\mathcal{L}_{\text{kd}} = \tfrac{T_d^2}{|\Omega|} \textstyle\sum_{\Omega} \mathrm{KL}\big(\,\mathrm{softmax}(\boldsymbol{z}^{\mathrm{t}}/T_d) \,\|\, \mathrm{softmax}(\boldsymbol{z}^{\mathrm{s}}/T_d)\big) \quad \text{(added 2026-10-19)}. \tag{13}$$

$\mathcal{L}_{\text{kd}}$ (Hinton et al. 2015) transfers the teacher's soft distribution (which cells are *nearly* right); the factor T_d^2 keeps the gradient scale independent of T_d. Teacher: a trained UFLD temporal model, frozen, on the same clip; a single-frame student reads only the last frame. Other terms: existence BCE with class weights (v0.2–v0.4), gate prior (v0.3), flow smoothness $\|\nabla \boldsymbol{u}\|_1$ (v0.5), auxiliary segmentation cross-entropy (official `use_aux`).
Code: `losses.py: softmax_focal_loss, coord_loss, temporal_consistency_loss, distillation_loss, VariantLoss`
Verified: KD is zero for identical logits, positive otherwise, zero with no annotated cell, and back-propagates into the student (`test_distillation_*`).

7 Evaluation metrics

Lane level (CULane protocol). Each lane is drawn as a polyline of width $w = \mathrm{round}(30 W_o/1640)$ (W_o = original width); $\mathrm{IoU}(i, j) = |M_i \cap M_j|/|M_i \cup M_j|$; one-to-one matching $\pi^\star = \arg\max_\pi \sum \mathrm{IoU}(i, \pi(i))$ (Hungarian); a match is TP if IoU $\geq \tau$ ($\tau = 0.5$). Over the test set

$$P = \tfrac{TP}{TP+FP}, \quad R = \tfrac{TP}{TP+FN}, \quad F_\beta = \tfrac{(1+\beta^2)PR}{\beta^2 P + R}. \tag{14}$$

On ELAS every frame has two lanes and every model predicts two, so each miss is one FP and one FN and $P = R = F_1$; the per-frame and anchor-level metrics separate the error types.
Code: `metrics.py: lane_mask, iou_matrix, match_lanes, prf, f_beta`

Frame level. FP/frame, FN/frame, frames with ≥ 1 FP or FN, and frame accuracy $= \Pr[\mathrm{FP} = \mathrm{FN} = 0]$.
Code: `evaluation/evaluator.py: _frame_rates`

Anchor level. A cell is TP if both GT and prediction say "lane" and $|\hat{c} - c| \leq \kappa$ cells ($\kappa = 10$ px); a wrong position counts as FP and FN. Existence errors on their own populations: false-alarm rate $= \mathrm{FP}_{\text{ex}}/(\mathrm{FP}_{\text{ex}}+\mathrm{TN})$, miss rate $= \mathrm{FN}_{\text{ex}}/(\mathrm{FN}_{\text{ex}} + N_{\text{both}})$; lateral MAE $= \frac{1}{N}\sum |\hat{x} - x|$ (original px), near and far (top quarter of the band, as ELAS's near/far error).
Code: `metrics.py: AnchorCounts, anchor_counts`

Ego lane (ELAS paper). Lane-centre deviation $|\hat{c} - c|$ at the bottom ROI row, in px and in % of the GT lane width.
Code: `evaluation/native.py: elas_center_counts`

Stability. Jitter $= \mathbb{E}|\hat{x}_t - \hat{x}_{t-1}|$ over cells present in consecutive frames. OpenLane-V: RVLD's flicker and missing rates over GT-matched lane pairs of consecutive frames (stable / flickering / missing).
Code: `metrics.py: temporal_jitter; evaluation/rvld_protocol.py`
Verified: compared line by line with the authors' `evaluate_temporal.py` and `evaluate_iou_laneatt.py`.

8 Output-level Bayesian filtering

State-space model. Per cell, state $\boldsymbol{m} = (x, \dot{x})^\top$, $\boldsymbol{A} = \begin{pmatrix} 1 & \Delta \\ 0 & 1 \end{pmatrix}$, $\boldsymbol{H} = (1\ 0)$, process noise of a piecewise-constant white acceleration of variance q, $\boldsymbol{Q} = q \begin{pmatrix} \Delta^4/4 & \Delta^3/2 \\ \Delta^3/2 & \Delta^2 \end{pmatrix}$. (The book's continuous Wiener velocity model gives $q \begin{pmatrix} \Delta^3/3 & \Delta^2/2 \\ \Delta^2/2 & \Delta \end{pmatrix}$; both are standard, q is tuned on validation.)

Kalman filter (Särkkä 2013, Theorem 4.2).

$$\boldsymbol{m}_k^- = \boldsymbol{A}\boldsymbol{m}_{k-1}, \qquad \boldsymbol{P}_k^- = \boldsymbol{A}\boldsymbol{P}_{k-1}\boldsymbol{A}^\top + \boldsymbol{Q}, \tag{15}$$

$$v_k = y_k - \boldsymbol{H}\boldsymbol{m}_k^-, \quad S_k = \boldsymbol{H}\boldsymbol{P}_k^-\boldsymbol{H}^\top + R_k, \quad \boldsymbol{K}_k = \boldsymbol{P}_k^-\boldsymbol{H}^\top S_k^{-1}, \tag{16}$$

$$\boldsymbol{m}_k = \boldsymbol{m}_k^- + \boldsymbol{K}_k v_k, \qquad \boldsymbol{P}_k = \boldsymbol{P}_k^- - \boldsymbol{K}_k S_k \boldsymbol{K}_k^\top. \tag{17}$$

Code: `evaluation/tracking.py: LaneKalmanTracker.update`
Verified: the code's scalar expressions equal these matrix products term by term: $P_{00}^- = P_{00} + 2\Delta P_{01} + \Delta^2 P_{11} + Q_{00}$, $P_{01}^- = P_{01} + \Delta P_{11} + Q_{01}$, $P_{11}^- = P_{11} + Q_{11}$; $K = (P_{00}^-, P_{01}^-)/S$; $P_{00} = (1-K_0)P_{00}^-$, $P_{01} = (1-K_0)P_{01}^-$, $P_{11} = P_{11}^- - K_1 P_{01}^-$ (since $\boldsymbol{K}S\boldsymbol{K}^\top = \boldsymbol{K}\boldsymbol{H}\boldsymbol{P}^-$).

Uncertainty-aware measurement noise (added 2026-10-19). Theorem 4.2 holds for a time-varying R_k. We set $R_k = \max(r,\ s^2\sigma_k^2)$ with σ_k^2 the network's own predictive variance (Section 2). The gain $K_k = P_{00}^-/(P_{00}^- + R_k)$ then drops when the frame is uncertain (occlusion, blur), so the filter trusts its prediction, and rises when the frame is sharp. The innovation is Gaussian, $v_k \sim \mathcal{N}(0, S_k)$, so the gate $|v_k| > g\sqrt{S_k}$ ($g = 3$: 0.27 % false re-initialisation under the model) replaces a fixed pixel threshold.
Verified: with the same jump in the measurement, a frame with $\sigma^2 = 400$ moves the estimate less than one with $\sigma^2 = 4$ (`test_adaptive_kalman_trusts_uncertain_frames_less`).

Link to the open problem. Learned gates trust the history without a calibrated measure of its reliability; this filter makes the weighting explicit and explainable. It is the output-level version of the uncertainty-weighted fusion proposed for the networks.

9 Ego-lane geometry, lane change, LDW and LKA

State of the ego lane. With boundary abscissas x_L, x_R at the reference row y_r and the camera column c_x (car axis): centre $c = \frac{1}{2}(x_L + x_R)$, width $w_\ell = x_R - x_L$, lateral offset $o = (c_x - c)/w_\ell$ ($o = 0$ centred, $o = \pm\frac{1}{2}$ centre on a line).

Heading from the vanishing point. For a pinhole camera over a flat, straight road, both boundaries are image lines through the vanishing point (v_x, v_y) of the road direction. Writing each boundary through its reference- and far-row points, $x_j(y) = x_j(y_r) + s_j(y - y_r)$, the intersection is

$$v_x = x_L + s_L \frac{x_R - x_L}{s_L - s_R}. \tag{18}$$

For a camera aligned with the road, $v_x = c_x$ (the principal point), whatever the lateral offset; a yaw ψ moves it by $f \tan\psi$. Hence $v_x - c_x$ measures heading.
Code: `ego.py: vanishing_x, ego_lane_state`

Proposition 1 (why the former cue was wrong)**.** Let $c(y)$ be the lane centre at row y. Since both boundaries pass through the vanishing point, so does their midline: $c(y) = v_x + (c(y_r) - v_x)\frac{y - v_y}{y_r - v_y}$. Therefore

$$c(y_f) - c(y_r) = (c(y_r) - v_x)\,\frac{y_f - y_r}{y_r - v_y}.$$

For an aligned car ($v_x = c_x$), $c(y_r) - c_x = -o\,w_\ell$, so the former "heading" $c(y_f) - c(y_r)$ is proportional to the lateral offset o, not to the heading. Its sign ($y_f < y_r$) made the term push the car away from the centre.

Verified: `test_heading_cue_is_yaw_not_offset`: an off-centre aligned car gives heading 0; a yawed car gives the expected 60 px.

Lane change. Smoothed offset $\bar{o}_t$ (moving average), lateral speed $\dot{o}_t \approx (\bar{o}_t - \bar{o}_{t-n}) f_s / n$, time to line crossing $\mathrm{TLC} = (\frac{1}{2} - |\bar{o}|)/(\mathrm{sign}(\bar{o})\dot{o})$. A manoeuvre starts after k consecutive frames with $|\bar{o}| \geq o_s$, $\mathrm{sign}(\bar{o})\dot{o} \geq v_s$ and $\mathrm{TLC} \leq \tau_s$, or when a boundary changes side of c_x; or (predicted lanes) when the offset flips sign by ≥ 0.35 lane widths, confirmed over 3 frames, which is a crossing toward the side the car was approaching. It ends after m centred frames. Planned: an interacting-multiple-model filter with "keep" and "change" dynamics (Toledo-Moreo and Zamora-Izquierdo 2009) instead of thresholds.
Code: `ego.py: LaneChangeDetector`
Verified: ground-truth lanes: 8/8 events, 8/8 directions, 0 false alarms; predicted lanes, unseen scenes: 4/4, 4/4.

LDW and LKA. The wheel reaches the line at $|o| = \frac{1}{2} - h$ (h = half the car width in lane widths): $\mathrm{TLC}_w = (\frac{1}{2} - h - |o|)/(\mathrm{sign}(o)\dot{o})$; warning if $\mathrm{TLC}_w \leq \tau_w$ or the margin is $\leq \mu$. Steering (Stanley-type, normalised): $\delta = \mathrm{clip}(-k_o\, o + k_\psi (v_x - c_x)/w_\ell, \pm\delta_{\max})$. Both are suppressed only by an active turn signal (either side).
Code: `ego.py: lane_keeping`

10 Robustness protocols

For a degradation $\mathcal{D}_\theta$ drawn with a fixed per-sample seed: *current* $\mathcal{C} = (I_1, \ldots, I_{T-1}, \mathcal{D}_\theta(I_T))$; *persistent* $\mathcal{C} = (\mathcal{D}_\theta(I_1), \ldots, \mathcal{D}_\theta(I_T))$ (same θ in every frame); *held-out* corruption families never used in training (cast shadow, glare). Training option `current_frame_persistent_share`: the share of degraded clips that use the persistent form.
Code: `data/transforms.py: degrade_current_frame; evaluation/robustness.py`

11 Statistics

Seeds $i = 1..n$ give paired differences $d_i = m_i^{\mathrm{model}} - m_i^{\mathrm{ref}}$. Mean and t-interval $\bar{d} \pm t_{0.975,n-1} s_d/\sqrt{n}$; exact two-sided Wilcoxon signed-rank test. With n pairs the smallest attainable two-sided p is $2/2^n$ (all signs agree), so $n \geq 6$ is needed for $p < 0.05$ and, with Holm over a family of m tests (step-down: $p_{(k)}^{\mathrm{adj}} = \max_{j \leq k} \min(1, (m-j+1)p_{(j)})$), $n \geq \lceil \log_2(2m/0.05) \rceil$; two primary comparisons need $n \geq 7$.
Code: `evaluation/stats.py: paired_comparisons, holm, min_attainable_p, seeds_needed`

12 Cost

MACs: convolution $C_{\mathrm{out}} H_{\mathrm{out}} W_{\mathrm{out}} (C_{\mathrm{in}}/g)\, k_h k_w$; linear $N\, d_{\mathrm{in}} d_{\mathrm{out}}$; multi-head attention $N_q d^2 + 2N_k d^2 + N_q d^2 + 2N_q N_k d$; anchor-row pooling $C(A h_f w_f + A w_f G)$. Streaming cost per frame = one backbone pass + fusion + head (history cached). Measured at 384×512: UFLD 7.12, light baseline 1.99, light v0.5 2.39 GMACs.
Code: `evaluation/efficiency.py: count_macs`

13 Planned: distance and time to collision

Flat road, camera height h_c, focal length f_y, horizon row v_0: a vehicle whose box bottom is at row v_b is at range $d = f_y h_c/(v_b - v_0)$. Error propagation: $\partial d/\partial v_b = -d^2/(f_y h_c)$, so a one-pixel error grows with d^2. A constant-velocity Kalman filter on $(d, \dot{d})$ gives the closing speed; time to collision $\mathrm{TTC} = d/(-\dot{d})$ for $\dot{d} < 0$, time headway $\mathrm{THW} = d/v_{\mathrm{ego}}$; warning thresholds follow FCW practice (ISO 15623, Euro NCAP). To be implemented with the vehicle (CIPO) detector and validated with CARLA's true ranges.

References

Dickmanns, E.D., Mysliwetz, B.D. Recursive 3-D road and relative ego-state recognition. IEEE TPAMI 14(2), 1992.
Hinton, G., Vinyals, O., Dean, J. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.
Jin, D., Kim, D., Kim, C.-S. Recursive video lane detection. ICCV 2023.
Lin, J., Gan, C., Han, S. TSM: Temporal shift module for efficient video understanding. ICCV 2019.
Qin, Z., Wang, H., Li, X. Ultra fast structure-aware deep lane detection. ECCV 2020.
Särkkä, S. Bayesian Filtering and Smoothing. Cambridge University Press, 2013.
Toledo-Moreo, R., Zamora-Izquierdo, M.A. IMM-based lane-change prediction in highways with low-cost GPS/INS. IEEE T-ITS 10(1), 2009.
Berriel, R.F. et al. Ego-Lane Analysis System (ELAS): dataset and algorithms. Image and Vision Computing, 2017.